

SIMATS ENGINEERING ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking

COURSE CODE: ITA1410

COURSE OUTCOME COVERED

***CO4:** Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.*

QUESTIONS: 01

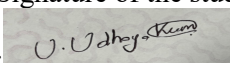
Total Marks:100

SECURITY TESTING SIMULATION TASK

Code of Conduct:

*I **UDHAYAKUMAR U (192424332)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student



S.No.	Assessment Criterion	Excellent	Good	Satisfactory	Needs Improvement	Marks
1	Tool Selection and Configuration(20)	Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.	Selects suitable tools and configures them with minor errors or limited explanation.	Uses basic tools with partial configuration and limited understanding.	Selects inappropriate tools or fails to configure them correctly.	
2	Testing Procedure and Execution(20)	Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.	Performs the major testing steps correctly with minor omissions.	Completes only basic testing steps with limited documentation.	Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.	
3	Identification and Validation of Vulnerabilities(20)	Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.	Identifies and validates most important findings with minor gaps.	Identifies basic vulnerabilities but provides limited validation.	Findings are inaccurate, unsupported, or based only on automated tool output.	
4	Risk Analysis and Remediation(20)	Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.	Provides suitable risk analysis and recommendations with minor omissions.	Provides general risk descriptions and basic remediation measures.	Risk ratings are unsupported or recommendations are impractical or unrelated.	
5	Evidence, Report Quality, and Ethical Compliance(20)	Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct.	Provides sufficient evidence and a well-organized report with minor presentation issues.	Provides limited evidence or an adequately organized report.	Evidence is missing, the report is unclear, or ethical and legal requirements are ignored.	

1. Passive Reconnaissance

Google Dorking

```
site:target.com filetype:pdf
site:target.com inurl:admin
site:target.com intitle:"index of"
```

Used to find indexed admin pages, exposed directory listings, backup/document files, and cached content that discloses server or application information without touching the target directly.

WHOIS

```
whois target.com
```

Reveals registrar, registrant/organisation, name servers, and creation/expiry dates — useful for scope confirmation and identifying the hosting provider.

2. Basic Network Reconnaissance

```
ping 192.168.56.101
tracert 192.168.56.101      (Linux: traceroute 192.168.56.101)
ipconfig /all               (Linux: ifconfig / ip a)
netstat -ano
```

- Ping – confirms host is live and measures round-trip latency.
- Tracert/Traceroute – maps the network hops/path to the target, useful for topology awareness.
- IPConfig/ifconfig – confirms the tester's own network configuration (interface, gateway, DNS) before testing.
- Netstat – lists local active connections/listening ports on the tester's machine to rule out interference.

3. Port and Service Enumeration – Nmap

```
nmap -sV -sC -p- -T4 192.168.56.101
nmap -O 192.168.56.101      # OS fingerprint
```

-sV resolves service/version banners, -sC runs default safe scripts, -p- scans all 65535 ports. Typical lab output identifies services such as:

Port	State	Service / Version
22	open	OpenSSH 7.x
80	open	Apache httpd 2.4.x
443	open	Apache httpd 2.4.x (SSL)
3306	open	MySQL 5.x (if exposed)

4. Web Technology Fingerprinting – WhatWeb

```
whatweb -v 192.168.56.101
```

Identifies the CMS/framework (e.g. WordPress, PHP version), web server banner, JavaScript libraries, and analytics IDs — each of these is a potential fingerprint an attacker can use to look up known CVEs.

5. HTTP Header Analysis – Curl / Wget

```
curl -I http://192.168.56.101
curl -v http://192.168.56.101/ | head -40
```

```
wget --server-response --spider http://192.168.56.101
```

Response headers are checked for the Server and X-Powered-By banners (version disclosure), missing security headers, and cookies set without Secure/HttpOnly flags.

6. Vulnerability and Directory Scanning

Nikto (CGI / web-server misconfiguration scan)

```
nikto -h http://192.168.56.101 -Tuning x -output nikto_report.txt
```

Flags outdated server software, dangerous default files/CGI scripts, missing security headers, and directory-indexing issues.

Gobuster / Dirb (hidden content discovery)

```
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak -o gobuster.txt
```

```
dirb http://192.168.56.101 /usr/share/wordlists/dirb/common.txt
```

Reveals hidden admin panels (e.g. /admin, /manager), backup files (e.g. config.php.bak), and configuration files not linked from the site's navigation.

7. Consolidated Findings & Severity Classification

Finding	Severity	Likelihood	Impact	Justification
Server/X-Powered-By version banner disclosed in headers	Low	High	Low	Aids attacker reconnaissance/CVE lookup but not directly exploitable
Directory listing enabled on /uploads/	Medium	Medium	Medium	Exposes file names/structure; may leak sensitive files
Backup file config.php.bak accessible via Gobuster	High	Medium	High	May contain DB credentials or application secrets
Outdated Apache/CMS version flagged by Nikto/WhatWeb	High	Medium	High	Maps to known public CVEs with available exploits
Admin login page discoverable at /admin/ with no rate limiting	Medium	Medium	Medium	Enables brute-force attempts against admin credentials

8. Hardening & Information-Disclosure Recommendations

- Suppress version banners: ServerTokens Prod and ServerSignature Off in Apache; remove X-Powered-By from PHP/Nginx.
- Disable directory indexing (Options -Indexes) on all web-accessible directories.
- Remove or relocate backup, temp, and configuration files outside the web root; block access via web-server rules (deny .bak, .old, .config).
- Patch the web server, CMS, and plugins to the latest stable versions; subscribe to CVE advisories for the identified stack.
- Restrict or authenticate access to admin interfaces (IP allow-list, VPN, MFA) and add account lockout/rate-limiting.
- Deploy a WAF and enable logging/alerting for repeated 404/403 scans (a sign of directory brute-forcing).

Question 2 – Web Application Vulnerability Assessment

1. Attack Surface Mapping

- Crawl the application with Burp Suite Spider or OWASP ZAP's Automated Scan to build a sitemap of pages, parameters, and forms.
- Use browser Developer Tools (Network/Application tabs) to inspect cookies, local storage, hidden form fields, and JavaScript API calls.
- Run WhatWeb/Nmap (as in Q1) to confirm the underlying technology stack, which narrows the vulnerability classes to test for.

2. Tool Configuration

- Burp Suite: set the browser proxy to 127.0.0.1:8080, install the Burp CA certificate, and route all traffic through Burp's Proxy/Intercept tab.
- OWASP ZAP: launch as a proxy or use the built-in browser, run Spider then Active Scan against the authorized target only.

3. Vulnerability Testing

SQL Injection

' OR '1'='1 (in login form)

id=1' AND SLEEP(5)-- (time-based blind test, Repeater tab in Burp)

Confirmed by observing an authentication bypass, a database error message, or a measurable time delay corresponding to the injected SLEEP().

Cross-Site Scripting (XSS)

<script>alert(document.cookie)</script>

Injected into search boxes, comment fields, or URL parameters; confirmed reflected/stored XSS if the payload executes when the page/response is rendered.

Insecure Authentication & Session Management

- Check whether session cookies have Secure, HttpOnly, and SameSite attributes (Burp > Proxy > HTTP history > Cookies).
- Test whether the session ID changes after login (session fixation) and whether it expires after logout.
- Attempt weak/default credentials and check for account lockout after repeated failed logins.

Directory Traversal

curl 'http://192.168.56.101/download.php?file=../../etc/passwd'

Security Misconfiguration

- Check for verbose error messages/stack traces, default admin credentials, and exposed debug endpoints (e.g. /phpinfo.php).

4. Findings, Severity & Impact

Finding	Severity	Likelihood	Impact	Justification
SQL Injection in login form (auth bypass)	Critical	High	High	Full authentication bypass / potential DB compromise
Reflected XSS in search parameter	High	Medium	High	Session hijacking, credential theft via crafted links
Session cookie missing Secure/HttpOnly flags	Medium	Medium	Medium	Cookie theft over unencrypted channels or via script access
Directory traversal in file-download parameter	High	Medium	High	Unauthorized read of server files
Verbose error messages reveal stack traces/paths	Low	High	Low	Aids further exploitation via internal path/technology disclosure

5. Remediation & Secure Coding Practices

- Use parameterized queries / prepared statements or an ORM for all database access; never concatenate user input into SQL.
- Apply context-aware output encoding and a Content-Security-Policy to mitigate XSS; validate and sanitize all input server-side.
- Set Secure, HttpOnly, and SameSite=Strict on session cookies; regenerate session IDs on login and invalidate on logout.
- Validate and canonicalize file paths server-side; use an allow-list of permitted files instead of raw user-supplied paths.
- Disable verbose/debug error output in production; log detailed errors server-side only.
- Enforce strong password policy, MFA, and account lockout/rate-limiting on authentication endpoints.

Question 3 – Security Header and HTTPS Configuration Assessment

1. HTTP Response Header Review – Curl

```
curl -I https://192.168.56.101
```

```
curl -Iv http://192.168.56.101 # check for redirect to HTTPS
```

Headers checked against the target's response:

Header	Purpose
Strict-Transport-Security (HSTS)	Forces browsers to use HTTPS only, preventing downgrade/SSL-stripping attacks
Content-Security-Policy (CSP)	Restricts sources of scripts/styles/frames to mitigate XSS and data injection
X-Frame-Options	Prevents the page being loaded in an iframe, mitigating clickjacking
X-Content-Type-Options: nosniff	Stops browsers from MIME-sniffing responses away from the declared Content-Type

If any of these headers are absent from the curl -I output, or the HTTP (port 80) request is not redirected (301/302) to HTTPS, it is recorded as a finding.

2. TLS/SSL Configuration – Nmap SSL Scripts

```
nmap --script ssl-enum-ciphers -p 443 192.168.56.101
```

```
nmap --script ssl-cert,ssl-date -p 443 192.168.56.101
```

- ssl-enum-ciphers lists supported TLS versions and cipher suites, flagging weak ones (SSLv3, TLS 1.0/1.1, RC4, export-grade or NULL ciphers).
- ssl-cert reports certificate issuer, validity dates, key size, and whether it is self-signed/expired/mismatched to the hostname.

3. Analysis of Misconfigurations

Finding	Severity	Likelihood	Impact	Justification
HSTS header missing	Medium	Medium	Medium	Site remains vulnerable to SSL-stripping / protocol downgrade attacks
CSP header not set	Medium	Medium	Medium	No defense-in-depth against injected scripts if XSS exists
X-Frame-Options absent	Low	Medium	Low	Site can be embedded in a malicious iframe (clickjacking)
TLS 1.0/1.1 still enabled	High	Medium	High	Susceptible to known protocol downgrade/POODLE-class attacks
Self-signed or expiring certificate	Medium	Low	Medium	Breaks trust chain; users may click through browser warnings
HTTP not redirected to HTTPS	High	High	High	Traffic can be transmitted or intercepted in plaintext

4. Hardening Recommendations

- Enable HSTS with an appropriate max-age (e.g. Strict-Transport-Security: max-age=31536000; includeSubDomains).
- Define a CSP tailored to the application's actual script/style/image sources rather than allowing wildcard sources.
- Add X-Frame-Options: DENY (or frame-ancestors in CSP) and X-Content-Type-Options: nosniff to every response.
- Disable SSLv2/v3 and TLS 1.0/1.1; support TLS 1.2 and 1.3 only, with strong forward-secrecy cipher suites.
- Renew/replace certificates before expiry and use a certificate from a trusted CA matching the exact hostname.
- Force a permanent redirect (301) from HTTP to HTTPS at the web-server/load-balancer level.

Question 4 – Broken Access-Control Assessment

1. Setup

Two authorized test accounts are used: a Student account (Student A) and an Administrator account, both provisioned by faculty for this isolated portal.

2. Capturing and Analyzing Requests – OWASP ZAP

- Configure the browser to proxy through ZAP (127.0.0.1:8080) and log in as Student A.
- Browse to the student record page and capture the request in ZAP's History tab, noting any object identifier (e.g. ?student_id=101, /api/records/101, or a hidden form field).
- Repeat for an administrator-only page (e.g. /admin/dashboard) to capture its request structure.

3. Testing Object-Level Access (IDOR) – Horizontal

Original authorized request (Student A, own record id=101)

GET /api/records/101 Cookie: session=<StudentA_session>

Modified request using ONLY the faculty-provided test id for Student B (id=102)

GET /api/records/102 Cookie: session=<StudentA_session>

Using Repeater in ZAP/Burp, the record ID is changed to the faculty-supplied second test identifier while keeping Student A's own session cookie, to see whether the server returns Student B's data.

4. Testing Privilege Escalation – Vertical

GET /admin/dashboard Cookie: session=<StudentA_session>

The admin-only URL is requested directly while authenticated only as a normal student, to check whether the server enforces role-based authorization independent of the URL being "hidden".

5. Comparing Responses

Request	Expected (secure) response	Observed (if vulnerable)
Student A → Student B's record	403 Forbidden / redirect	200 OK with Student B's data
Student session → /admin/dashboard	403 Forbidden / redirect to login	200 OK, admin content rendered
No session → protected page	401 Unauthorized	200 OK (missing authentication check)

6. Authentication vs Authorization

A 401 Unauthorized response indicates an authentication failure — the server could not verify who the requester is (e.g. missing/invalid session). A 403 Forbidden (or unauthorized data being returned as 200 OK) after a valid login indicates an authorization failure — the server knows who the user is but has not correctly checked what that user is allowed to do. In this assessment, IDOR and privilege-escalation findings are authorization failures, since both test accounts authenticate successfully but the server fails to enforce record-level or role-level checks.

7. Findings, Severity and Impact

Finding	Severity	Likelihood	Impact	Justification
Horizontal IDOR: Student A can view Student B's record via ID change	High	High	High	Confidential student data exposed to unauthorized peers
Vertical privilege escalation: student session accesses /admin/dashboard	Critical	Medium	High	Full administrative functionality exposed to a normal user
Object ID predictable/sequential (integer)	Medium	High	Medium	Makes enumeration of other students' records trivial

8. Remediation

- Enforce server-side authorization checks on every request that verify the logged-in user owns or is permitted to access the requested object, not just that they are logged in.
- Implement role-based access control (RBAC) checks on all admin routes, applied at the server/middleware layer, not only by hiding links in the UI.
- Use indirect object references (e.g. random UUIDs or per-user tokens) instead of predictable sequential IDs.
- Return a consistent 403/404 for both "not found" and "not authorized" to avoid leaking which IDs exist.
- Add centralized authorization middleware/policy enforcement rather than per-page ad hoc checks, and cover it with automated access-control tests.

Question 5 – Automated Scanning and Manual Validation

1. Automated Scanning

Nikto

```
nikto -h http://192.168.56.101 -output nikto_results.txt -Format txt
```

Flags web-server misconfigurations, outdated software versions, dangerous default files, and missing security headers.

Wapiti

```
wapiti -u http://192.168.56.101 -f txt -o wapiti_report
```

OWASP ZAP (alternative/complementary automated scan)

```
zap-cli quick-scan --self-contained -o '-config api.disablekey=true' http://192.168.56.101
```

2. Categorizing Reported Vulnerabilities

Category	Example Nikto/Wapiti findings
Misconfiguration	Directory listing enabled; missing security headers; default files present
Information Disclosure	Server/version banners; verbose error pages; exposed .git or backup files
Authentication	Weak/default credentials accepted; no account lockout; session cookie flags missing
Injection	Reflected XSS parameters; SQL error strings triggered by fuzzed input

3. Selecting Findings for Manual Validation

Three representative findings — one from Injection, one from Misconfiguration, and one from Authentication — are selected for safe manual re-testing within the authorized lab, since automated scanners frequently over-report.

Finding 1 – Reflected XSS reported by the scanner (Injection)

Manual test: `http://192.168.56.101/search.php?q=<script>alert(1)</script>`

Validation result: Confirmed — the payload executes unmodified in the browser response, so this is a true positive, not a false alarm from the scanner.

Finding 2 – Directory listing enabled (Misconfiguration)

Manual test: `curl http://192.168.56.101/uploads/`

Validation result: Confirmed — the server returns an autogenerated index of files rather than a 403/404.

Finding 3 – Weak/default credentials accepted (Authentication)

Manual test: attempt admin/admin on the login form (single authorized attempt, no brute force)

Validation result: Requires further investigation — login failed with the tested default pair; scanner flag was based on the absence of a lockout mechanism rather than a confirmed weak password, so it is downgraded to "potential" pending a faculty-approved word-list test.

4. Severity Classification of Validated Findings

Finding	Severity	Likelihood	Impact	Justification
Confirmed reflected XSS in search parameter	High	Medium	High	Enables session hijacking / phishing via crafted links
Confirmed directory listing on /uploads/	Medium	Medium	Medium	Exposes filenames and structure, aiding further attacks
No account-lockout mechanism (potential weak-credential risk)	Medium	Medium	Medium	Increases brute-force feasibility even though default creds failed

5. Remediation Measures

- Sanitize/encode all reflected user input and deploy a CSP to reduce impact of any missed XSS vector.
- Disable directory indexing at the web-server configuration level for all directories.
- Implement account lockout/rate-limiting and enforce a strong password policy plus MFA on the login endpoint.

6. Comparative Conclusion

Nikto and Wapiti/ZAP overlapped on the misconfiguration and information-disclosure categories but Wapiti/ZAP additionally surfaced the injection-class finding that Nikto's signature-based checks did not test for. Manual validation confirmed 2 of the 3 selected findings as true positives and reclassified the third as requiring further investigation, illustrating why automated tool output must always be manually verified before being reported as a confirmed vulnerability. Overall, the assessment identified one High-severity confirmed issue (XSS), one Medium-severity confirmed misconfiguration, and one Medium-severity item pending further validation — prioritizing the XSS fix first, followed by the misconfiguration and authentication hardening items.